

Aurora 战队雷达站传承文档

RoboMaster RMUC 2026 — 从 T-DT 开源到上场

2026 赛季雷达负责人

2026 年 9 月

摘要

本文档记录 2026 赛季我们如何拿到东北大学 T-DT 实验室的 2025 赛季开源雷达代码，一步步改造为 Aurora 战队 RMUC 2026 的上场系统：改了什么、为什么改、改完什么效果、还剩什么坑。面向 2027 赛季接手的同学。

目录

1 起点：拿到的 T-DT 开源是什么样的	1
1.1 来源与定位	1
1.2 原版代码结构（我们拿到的）	1
2 改造总览：代码量翻了几倍	2
3 逐模块改造记录	2
3.1 相机驱动（从无到有）	2
3.2 检测 detect：加超分、加 3D 框、加 5 槽输出	3
3.3 融合 kalman_filter：从简单跟踪到身份锁定 + 盲区猜点	3
3.4 标定 calibrate：按 2026 场地重做 5 点流程	3
3.5 解算 resolve：加 HeightGrid 地形修正	3
3.6 配准 localization：CPU GICP → CUDA ICP	3
3.7 串口通信：从零自研（上场必须）	4
3.8 debug_map：加盲区猜点、双倍易伤全自动	4
3.9 工具链：控制面板、录包工具、SR 训练	4
4 2026 上场效果与遗留问题	4
4.1 能跑，识别准，但易伤很差	4
4.2 为什么双倍易伤一次都没开成	5
4.3 两场未启动事故	5
5 2027 赛季注意事项	5
A 关键文件索引	5

1 起点：拿到的 T-DT 开源是什么样的

1.1 来源与定位

上游是东北大学 T-DT 实验室的 2025 赛季雷达开源 (T-DT_Radar-main, MIT 协议, 技术报告见 RM 论坛 <https://bbs.robomaster.com/article/803954>)。README 里明确一句话：“**本项目提供了除串口、相机驱动、模型训练外雷达站的全部功能**”。也就是说，感知 + 融合 + 标定 + 地图可视化是现成的，但**与裁判系统通信、取图、推理模型**三块是空的，而这三块恰恰是上场最关键的部分。

原版硬件平台：Livox Avia + 海康 CH-120-10UC + i7-12700KF + RTX A4000×2。我们的硬件：Livox Mid-70 + 海康 6MP + i7 + RTX 3050M（算力弱很多，直接影响推理方案选择）。

1.2 原版代码结构（我们拿到的）

- src/tdt_vision/：相机标定 (calibrate)、检测 (detect)、解算 (resolve)、推理 (infer)、工具 (utils)
- src/lidar/：localization(GICP 配准)、cluster(欧式聚类)、dynamic_cloud(动态点)
- src/fusion/：kalman_filter(卡尔曼融合跟踪)、debug_map(地图可视化)
- src/interface/：自定义消息 (DetectResult/MatchInfo/Radar2Sentry 等)
- src/livox_driver/：Livox 驱动（原版自带，我们直接沿用）
- src/utils/：rosbag_player 回放工具、onnx2trt 转换脚本

模块化做得很清楚，低耦合，确实易于扩展。但每块都是“骨架”：detect.cpp 只有 334 行、kalman_filter.cpp 206 行、debug_map.cpp 285 行、resolve.cpp 141 行、localization.cpp 187 行。**核心逻辑都在，但只够跑通“实验室回放”，离上场差很远。**

2 改造总览：代码量翻了几倍

文件	T-DT 原版	本项目	核心改动	为什么改
detect.cpp	334	999	+SR 超分、3D 框、5 槽位输出、机器人号映射	算力弱要超分提精度；上场要按规则发指定槽位
kalman_filter.cpp	206	1488	+Hungarian 匹配、id_score 身份置信、行为猜点	原版只做简单跟踪，上场要持续锁定 + 盲区猜
debug_map.cpp	285	728	+ 盲区猜点、双倍易伤全自动触发、决策包	原版只显示，上场要自动开易伤
resolve.cpp	141	329	+HeightGrid 地形高度修正	场地有坡，平面透视变换不够准
localization.cpp	187	483	CPU GICP → CUDA ICP	CPU 配准太慢，GPU 提速
cluster.cpp	84	250	+ 更多聚类参数、飞镖/空中过滤	2026 车型变化
calibrate.cpp	190	451	+ 键盘微调、5 点流程重写	原版标定流程不顺手
完全新增的模块：				
radar_serial_node.cpp	0	新增	裁判系统串口 (0x0305/0x0301/0x0303)	原版没有，上场必须
camera/ (hik.cpp)	0	新增	海康 SDK 相机驱动	原版没有，要自己写
rm_control_panel	0	新增	Qt 控制面板 (改 team/标定/录包)	原版只能改文件重启
databag_tool	0	新增	自研录包工具	原版只有回放，没有专门录包
create_sr_onnx.py	0	新增	SR 超分模型训练/转换脚本	原版无超分

表 1: 从 T-DT 开源到上场，核心文件代码量变化。

3 逐模块改造记录

3.1 相机驱动（从无到有）

原版没有相机驱动 (README 说的” 不含相机驱动”), 只有 camera_image 这个 topic 假设图已经来了。我们接入海康 MV-CS060 的 SDK (src/tdt_vision/camera/MV_SDK), 写 hik.cpp 封装取图、曝光控制、内参加载, 发布到 camera_image。夜场曝光参数单独存 config/camera_driver_night.yaml。

3.2 检测 detect：加超分、加 3D 框、加 5 槽输出

原版 334 行只做 YOLO 检测 + 装甲板分类。我们 RTX 3050M 算力远不如原版 A4000×2，直接跑 1280 输入太慢，所以：

1. **加 SR 超分**(15 处相关代码): 先用小分辨率 YOLO 快检, 再对 ROI 超分辨率放大, 在算力弱的卡上提高小目标识别率。训练脚本 `utils/create_sr_onnx.py`、转换 `utils/pb2onnx_sr.py`。
2. **加 3D 框**: 用外参把检测框投回 3D, 在 `detect` 窗口叠加蓝色长方体, 直观看出标定准不准(原版只出 2D 框)。
3. **加 5 槽位输出**: 按 2026 规则把装甲板数字映射到槽位 (1→ 英雄、2→ 工程、3→ 步兵 3、4→ 步兵 4、6→ 哨兵, 2026 无步兵 5 无空中), 填到 `DetectResult` 的 `red_x/blue_x` 数组。这里有个大坑见 §5.1: 同号覆盖。

3.3 融合 `kalman_filter`: 从简单跟踪到身份锁定 + 盲区猜点

原版 206 行只做基础卡尔曼跟踪。我们扩到 1488 行:

- **Hungarian 匹配**: KF 轨迹与新检测做二分图匹配, 避免”同一辆车配到两条轨迹”。
- **id_score 身份置信**: 每条轨迹维护一个”这辆车到底是谁”的置信度, 装甲板数字稳定才赋值, 避免偶发误分类直接改写身份。
- **行为分类猜点**: 车进盲区后不丢轨迹, 按位置 + 速度推断行为(CHARGING/TUNNEL/OUTPOST/ASSEMBLY 四类) 猜它在哪, 恢复后接管。
- **发布 /radar2sentry**: 把融合后的 5 槽坐标发给串口节点。

3.4 标定 `calibrate`: 按 2026 场地重做 5 点流程

在原版 5 点标定(Enter 开始、wasd 微调)基础上扩到 451 行: 标定点按 2026 场地更新(我方堡垒 → 前哨 → 敌方基地 → 敌方前哨 → 中央高塔), 外参按红蓝分色保存 `config/{red,blue}/out_matrix_*`, 配合 `detect 3D` 框直观验证贴合度。关键实战教训: 雷达每场被搬动过外参就失效, 必须每场间重标。

3.5 解算 `resolve`: 加 `HeightGrid` 地形修正

原版 141 行只做平面透视变换。RMUC 场地不是平的(有 43 度坡、高低台), 直接平面投会偏。我们加 `HeightGrid`: 预计算 `config/map/field_mesh.bin` 网格高度, 把像素投到世界坐标时用网格查实际地面高度修正, 329 行。

3.6 配准 `localization`: CPU GICP → CUDA ICP

原版 187 行用 CPU GICP 配准, 我们 483 行改成 CUDA ICP (`cuda_icp.h`)。原因还是算力: CPU 配准在前几帧收敛慢, 场上没法等。GPU 化后前几帧即可对齐。地图 `config/map/map.pcd` 换成 2026 场地扫描点云。

3.7 串口通信: 从零自研(上场必须)

这是原版完全没有的部分, 也是上场最关键、2026 出问题最多的部分。自研 `radar_serial_node.cpp`:

- 0x0305 小地图上传：6 敌 + 6 己方坐标 (48B)， $\leq 5\text{Hz}$ ，cm 单位， $x=y=0$ 视为未发送。标记精度： $<0.8\text{m}$ 计 +1， $0.8\sim 1.6\text{m}$ 计 +0.5， $\geq 1.6\text{m}$ 计 -0.8。
- 0x0301 + 0x0121：双倍易伤决策包（每局 2 次，每次 30s）。
- 0x0303：下行解析（比赛状态、HP）。
- 0x0001/0003/0101/0105/020C/020E $\rightarrow$ /match_info:0x020C = 标记成功位, 0x020E bit0-1 = 机会数、bit2 = 激活中。

串口节点本身只是搬运工，发几辆车完全由上游 detect/kalman 决定。2026 的核心问题就在这里 (§5)。

3.8 debug_map：加盲区猜点、双倍易伤全自动

原版 285 行只做地图显示。我们 728 行加：

- **盲区猜点**：车进盲区时在地图上按行为分类显示预测位置。
- **双倍易伤全自动触发**：条件 = 机会数 $>$ 已触发数且场上有标记且非激活 $\Rightarrow$ 自动发 /radar/decision_request 手动兜底：小地图窗口按 v 键。

3.9 工具链：控制面板、录包工具、SR 训练

原版只有 rosbag_player 回放和 onnx2trt 转换。我们新增：

- rm_control_panel (Qt)：比赛时改 team、触发标定、开关录包，不用改文件重启。
- databag_tool：自研录包，按场次自动命名到 bags/。
- create_sr_onnx.py / pb2onnx_sr.py：SR 超分模型训练 + 转 onnx。

4 2026 上场效果与遗留问题

4.1 能跑，识别准，但易伤很差

局均易伤约 475s/局（较差）。回放中识别和定位本身跑得很准，**问题不出在感知，出在”往裁判系统发什么”**。用 tools/analyze_vuln_bag.py 解码回放包：

对局	平均发车数	≥ 3 辆占比	标记成功 (0x020C)	双倍易伤激活
浙大 1 红	2.26	29%	0 次	0 秒
浙大 2 红	3.23	68%	0 次	0 秒
中国石油大学红	1.20	0%	0 次	0 秒

表 2：区域赛回放统计。整个赛季双倍易伤一次都没开成。

4.2 为什么双倍易伤一次都没开成

因果链：

- 1. detect 填槽时两车若得到相同 number，**后写直接覆盖先写** (§3.2 的 5 槽输出大坑)
- 2. 看得见车但没识别出装甲板数字 ⇒ number=-1 ⇒ 整车不发
- 3. 结果每帧只发 1~3 辆（浙大 1 平均 2.26 辆，99% 时间 ≤3 辆）
- 4. 发出去的坐标精度不足，按规则 ≥1.6m 倒扣分
- 5. 整场 0x020C 标记成功位**从未出现上升沿**
- 6. 0x020E 机会数恒为 0 ⇒ 自动触发条件永不满足 ⇒ **双倍易伤 0 秒**

根因不是双倍易伤按钮逻辑本身，而是发车少 + 精度差让标记机制空转。

4.3 两场未启动事故

区域赛有一局雷达完全没能启动，双倍易伤自然全程未开。归因：3 分钟准备时间内接线/配置未完成，且没有强制赛前自检。对策已写入 docs/preflight_check.md（赛前六项验证清单）。

5 2027 赛季注意事项

- 1. **车型变化**：2027 赛季少一个机器人。需改：robot_slots.h 槽位表、radar_serial_node.cpp 的协议映射、armor_class_to_slot 映射、0x0003 HP 表下标。
- 2. **重装/新型装甲板**：若 2027 装甲板样式有变化，现有 classify.onnx 和 armor_yolo 很可能不能直接用，训练数据里没有新样式，需尽早收集重训。
- 3. **优先修复**：P1 同槽覆盖 → P2 KF 轨迹直出数据源（不依赖装甲板数字才发车）→ P3 置信门控（发偏了不发，x=y=0 视为未发送）。
- 4. **流程**：赛前强制跑 preflight 清单 + 回放判据达标才上场（用 tools/analyze_vuln_bag.py，平均发车数 ≥3.5、≥3 辆占比 ≥80%）。

A 关键文件索引

文件	作用（相对 T-DT 原版的改动）
src/tdt_vision/camera/hik.cpp	新增 ：海康相机驱动
src/tdt_vision/detect/src/detect.cpp	+SR 超分、3D 框、5 槽输出（原 334→999 行）
src/tdt_vision/resolve/src/resolve.cpp	+HeightGrid 地形修正（141→329 行）
src/tdt_vision/calibrate/src/calibrate.cpp	点云流程重写（190→451 行）
src/lidar/localization/src/localization.cpp	ICP GICP→CUDA ICP（187→483 行）

src/lidar/cluster/src/cluster.cpp	+ 车型过滤 (84→250 行)
src/fusion/kalman_filter/src/kalman_filter_hungarian/id_score/猜点 (206→1488 行)	
src/fusion/kalman_filter/src/radar_sentry/新增: 裁判系统串口	
src/fusion/debug_map/debug_map.cpp	+ 猜点/双倍易伤自动 (285→728 行)
src/utls/rm_control_panel/	新增: Qt 控制面板
src/utls/databag_tool/	新增: 自研录包工具
tools/analyze_vuln_bag.py	新增: 回放易伤复盘统计
docs/preflight_check.md	赛前六项验证清单
README_DETAIL.md	启动/接线/标定/比赛日流程
config/radar_runtime.yaml	team 红蓝 / 串口 / 回放配置入口

B 话题速查

话题	类型	含义
camera_image	Image	相机原图 (原版无驱动, 本项目新增)
/detect_result	DetectResult	YOLO 检测 + 装甲板分类 + 5 槽
/resolve_result	DetectResult	像素 → 世界坐标 + HeightGrid
/kalman_detect	DetectResult	KF 融合输出
/radar2sentry	Radar2Sentry	串口主链输入 (5 槽敌 + 己坐标)
/match_info	MatchInfo	比赛状态 (self_color/marks/ultimate/HP)
/radar/decision_request	UInt8	双倍易伤触发计数
/radar/tx_raw	UInt8MultiArray	串口发送帧回放 (调试用)